

Fiche d'aide à l'examen

TAA – Partie Deep Learning

Ce PDF a pour objectif de réunir les concepts découverts pendant le cours de Deep Learning. **Vous aurez une version imprimée de cette fiche à l'examen.**

1 Bases du Deep Learning: optimisation

Objectif général: trouver le minimum de la *loss*. On veut trouver les *paramètres* θ de la fonction f_θ pour minimiser une *fonction de loss* avec un *dataset* $\mathcal{D}$: $\arg \min_{\theta} \mathcal{L}(f_\theta, \mathcal{D})$.

Imaginez un paysage montagneux: la position (x, y) dans le paysage est la valeur du vecteur paramètres θ , et l'altitude en cette position est la valeur de la *loss* $\mathcal{L}(f_\theta, \mathcal{D})$. On cherche la position optimale $\theta^* = (x^*, y^*)$ à laquelle l'altitude est la plus basse.

La fonction de *loss* $\mathcal{L}$ et la fonction f_θ (réseau de neurones) peuvent prendre plusieurs formes, vous trouverez plusieurs exemples ci-dessous.

Méthode générale: Descente de Gradient. On va chercher θ^* à l'aide de la *Descente de Gradient*. A toute étape t avec paramètres θ^t , si on peut trouver une petite variation $\Delta\theta^t$ qui va réduire un peu $\mathcal{L}$, il suffit de faire un petit pas dans cette direction: $\theta^{t+1} = \theta^t + \eta\Delta\theta^t$. Dans l'analogie de la montagne, à partir d'une position (x^t, y^t) cela correspond à faire un pas pour descendre la pente, et se retrouver en (x^{t+1}, y^{t+1}) légèrement plus bas dans le paysage ($\mathcal{L}$).

Le *gradient* de $\mathcal{L}$ indique presque ce que l'on veut: pour chaque paramètre θ_i^t dans θ^t , le gradient $\partial\mathcal{L}/\partial\theta_i^t$ indique comment modifier θ_i^t pour augmenter $\mathcal{L}$. Il suffit donc de “descendre” en suivant le gradient négatif: $\theta_i^{t+1} = \theta_i^t - \eta\partial\mathcal{L}/\partial\theta_i^t$.

Backpropagation. Dans le contexte du Deep Learning, l'algorithme pour calculer $\partial\mathcal{L}/\partial\theta_i^t$ s'appelle la *Backpropagation* car il applique la “chain rule” de dérivation en partant de l'output et en remontant le réseau f_θ à l'envers (backwards) pour chaque θ_i^t couche par couche (voir Section 2).

Optimisation, régularisation. Il existe plusieurs méthodes pour améliorer la descente de gradient (optimisateurs avancés avec momentum et learning rate η adaptatif, méthodes de régularisation comme Dropout ou régularisation L2) qui ne seront **pas demandés dans les questions**. En parler dans le bon contexte peut tout de même aider votre note si c'est pertinent.

Données et apprentissage supervisé. Le dataset $\mathcal{D}$ est à minima constitué de *données* $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$ où chaque donnée $\mathbf{x}_i$ est multi-dimensionnelle, par exemple un vecteur ($\mathbf{x}_i \in \mathbb{R}^n$), une image (donc une matrice 2 ou 3D *largeur* $\times$ *hauteur* $\times$ *channels*, $\mathbf{x}_i \in \mathbb{R}^{w \times h \times c}$), du texte (généralement une matrice: un ensemble de h mots, chacun représenté comme un vecteur de taille n , $\mathbf{x}_i \in \mathbb{R}^{h \times n}$). On divise généralement le dataset en *training* et *testing* pour contrôler la généralisation.

Dans le cadre de l'apprentissage *supervisé*, on a accès à des *labels* $\mathbf{y}_i$: $\mathcal{D} = \{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$ qui correspondent à la “réponse à la question”: identifier un objet sur une image (*classification* d'un élément parmi plusieurs choix), la taille/poids d'un objet à prédire (*régression*: trouver une ou plusieurs valeurs continues)...

Apprentissage auto-supervisé. L'apprentissage *auto-supervisé* s'affranchit de labels, et cherche à extraire des informations intéressantes de n'importe quelle donnée $\mathbf{x}_i$, généralement sous la forme d'un *vecteur latent* $\mathbf{z}_i = f_\theta(\mathbf{x}_i) \in \mathbb{R}^d$, où d est un nombre de dimensions arbitraire. Cela passe quasiment toujours par l'apprentissage de *tâches prétexte* où l'on utilise un deuxième morceau de réseau q_ϕ qui prend $\mathbf{z}_i$ en entrée et cherche à l'utiliser pour résoudre des tâches facilement extrayables des données pures. Par exemple: un auto-encodeur qui cherche à reconstruire $\mathbf{x}_i$ à partir de $\mathbf{z}_i$; un modèle de langage qui cherche à prédire le prochain mot de la phrase ou le mot caché; un modèle de vision qui cherche à remplir les trous dans une image. On parle généralement d'*encodeur* pour f_θ et de *décodeur* pour q_ϕ . Chacun cherche à trouver ses paramètres (resp. θ et ϕ) pour minimiser la *loss*. Voir Section 2.2 pour l'architecture.

2 Architectures

2.1 Couches paramétriques fondamentales

Décomposition en couches. L'idée principale du Deep Learning est de décomposer la fonction f_{θ} en une séquence de couches f^l avec $f_{\theta}(\mathbf{x}) = \mathbf{x} \rightarrow f^1(\mathbf{x}) \rightarrow f^2(f^1(\mathbf{x})) \rightarrow \dots \rightarrow f^{l-1}(\cdot) \rightarrow f^l(\cdot)$. Chaque couche est potentiellement composée de ses propres paramètres $\theta^{(l)}$, dans ce cas on pourra parler d'une couche *paramétrique*. L'idée est de décomposer la tâche en *couches d'abstraction*: image de millions de pixels $\rightarrow$ formes générales $\rightarrow$ présence d'yeux, nez, bouches $\rightarrow$ présence de visage $\rightarrow$ détection d'être humains.

Tant que toutes les opérations des couches sont dérivables, on peut appliquer la Backpropagation et donc la Descente de gradient: on peut trouver un minimum de la fonction de loss. Il y a deux éléments principaux à savoir sur une couche: l'opération réalisée et les paramètres appris.

Couche Linéaire / Fully Connected / MLP. La couche la plus générique, qui prend en entrée un vecteur et le transforme à l'aide d'une multiplication matricielle + vecteur de bias (opération dite *linéaire*). Comme des opérations linéaires successives ne peuvent pas modéliser autre chose que des opérations linéaires, il faut ajouter une fonction d'*activation* non-linéaire comme la sigmoid, tanh ou ReLU, notée σ . On peut considérer l'activation comme une couche distincte ou comme partie de la Fully Connected. Cela donne:

- $f^l(\mathbf{x}) = \sigma(W^l \mathbf{x} + \mathbf{b}^l)$, qui ressort un vecteur (taille d avec $W^l \in \mathbb{R}^{n \times d}, \mathbf{b}^l \in \mathbb{R}^d$).
- Paramètres (éléments appris par la backprop): tous les éléments de W^l et $\mathbf{b}^l$

Couche Convolutionnelle. Spécialisée pour les applications en image, elle prend en entrée des images (taille *largeur* $\times$ *hauteur* $\times$ *channels* par exemple RGB=3 channels; $\mathbf{x} \in \mathbb{R}^{w \times h \times c}$). Elle y applique des *filtres* (ou *kernel*) de taille $k \times k \times c$ (par exemple $3 \times 3 \times c$) qui cherchent à repérer des patterns dans l'image.

- On "applique" le filtre K sur une sous-partie $k \times k \times c$ de l'image I en faisant la somme de chacun des produits, $\sum_i K_i * I_i$. Chaque "application" ressort une valeur réelle, et lorsqu'on répète l'opération sur toutes les sous-parties de l'image, on obtient une image 2D. Lorsque l'on répète l'opération pour D filtres, on obtient une image de D channels en sortie.
- Paramètres (éléments appris par la backprop): D filtres de taille $k \times k \times c$.

Couche d'Attention. Spécialisée à la base pour le langage, elle prend en entrée une matrice $\mathbf{x}$ de taille $h \times n$, (h le nombre de tokens dans la séquence / prompt, n la taille de chaque token (vecteur)). **Le calcul exact d'attention n'est pas nécessaire à comprendre pour l'exam.**

2.2 Notion de Bloc et d'Architecture

Bloc de couches. Très souvent les couches sont réunies en *Blocs* que l'on répète, qui utilisent plusieurs couches paramétriques ci-dessus mais aussi des couches pour aider à la backpropagation (fonctions d'activations, dropout, normalisation, skip connection). Par exemple:

- Bloc ResNet: $\mathbf{x} \rightarrow \text{Conv } 3 \times 3 \rightarrow \text{Batch Norm} \rightarrow \text{ReLU} \rightarrow \text{Conv } 3 \times 3 \rightarrow \text{Batch Norm} \rightarrow \text{Résiduel (addition } +\mathbf{x}) \rightarrow \text{Fonction d'Activation}$. Ce Bloc peut être répété des dizaines voire centaines de fois!
- Bloc Transformer: $\mathbf{x} \rightarrow \text{Multi-Head Attention} \rightarrow \text{Résiduel (addition } +\mathbf{x}) \rightarrow \text{Layer Norm} \rightarrow \chi \rightarrow \text{Couche Linéaire} \rightarrow \text{Fonction d'Activation} \rightarrow \text{Couche Linéaire} \rightarrow \text{Résiduel (addition } +\chi) \rightarrow \text{Layer Norm} \rightarrow \text{Dropout}$.

Architecture, vecteur latent. Une *architecture* d'un réseau est alors la suite d'opérations permettant d'aller de l'input original $\mathbf{x}$ à la sortie du réseau, $f_{\theta}(\mathbf{x})$. Pour les réseaux profonds actuels, elle est souvent composée de plusieurs Blocs:

- Architecture ResNet: $\mathbf{x} \rightarrow \text{Bloc ResNet (15 fois)} \rightarrow \text{Couche Linéaire}$, par exemple de taille 10 pour identifier 10 classes.

- Architecture Transformer (type GPT): $\mathbf{x} \rightarrow$ Bloc Transformer (15 fois) $\rightarrow$ Couche Linéaire, par exemple pour prédire le token suivant (taille n).
- Fonctionnement général encodeur-décodeur (peu importe le type de donnée, “Bloc” générique):
 - Encodeur: $\mathbf{x} \rightarrow$ Bloc (M fois) $\rightarrow$ (couche Linéaire possiblement $\rightarrow$) vecteur *latent*¹ $\mathbf{z}$
 - Décodeur: $\mathbf{z} \rightarrow$ Bloc (M fois, souvent moins) $\rightarrow$ (couche Linéaire possiblement $\rightarrow$) sortie désirée ($\hat{\mathbf{x}}$ pour un auto-encodeur)
- on peut voir presque tous les réseaux comme une sorte d’architecture “encodeur-décodeur” souvent avec un décodeur très minimal: par exemple réseau de classification $\mathbf{x} \rightarrow$ Bloc (M fois) $\rightarrow$ couche Linéaire $\rightarrow$ sortie: c classes; ressemble à encodeur + décodeur d’une couche linéaire.

3 Exemples à travers les papiers des Présentations

Pour un problème et une approche donnés il y a deux informations principales à savoir: l’architecture et la loss utilisées.

Auto-encodeur MNIST: Pas présenté mais vu avec BioInfo. On cherche à reconstruire aussi fidèlement que possible l’input $\mathbf{x}$ en passant par un *espace latent* de dimension plus petite, $\mathbf{z} \in \mathbb{R}^d$.

- Architecture: encodeur + décodeur classique (voir 2.2), par exemple pour une image: encodeur: image $\mathbf{x}$ en entrée $\rightarrow$ Blocs ResNet $\rightarrow$ Flatten & Couche Linéaire $\rightarrow \mathbf{z}$. Décodeur: vecteur $\mathbf{z} \rightarrow$ Reshape $\rightarrow$ Blocs ResNet $\rightarrow$ image reconstruite $\hat{\mathbf{x}}$.
- Loss: pour chaque élément x_i de l’input (par exemples les pixels d’une image), loss MSE (mean squared error): $\mathcal{L} = \frac{1}{n} \sum_i (x_i - \hat{x}_i)^2$. On cherche simplement à minimiser l’écart entre réalité et reconstruction (regression). Ceci est la loss pour un input, faire la moyenne sur b éléments de batch pour la Descente de Gradient.

YOLO: On cherche à prédire tous les éléments d’une image sous forme de boîtes + classification de l’objet dans la boîte (*bounding box*) (on a accès à des données très labélisées). On découpe l’image en $T \times T$ cellules (e.g. 7×7) et pour chaque cellule on prédit (*i*) le centre de l’objet dans la cellule (x, y) , (*ii*) la taille de la boîte pour l’objet qui se trouve centré dans la cellule (h, w) , (*iii*) la classe de l’objet dans la cellule (dimension c), (*iv*) la confiance qu’il y a bien un objet dans cette cellule (p entre 0 et 1). Il y a donc $T \times T \times (5 + C)$. En pratique il est possible de prédire B bounding boxes plutôt qu’une seule.

- Architecture: Type ResNet. En gros encodeur $\rightarrow$ sortie (“décodeur” minimal): image $\mathbf{x}$ en entrée $\rightarrow$ Blocs ResNet $\rightarrow$ prédiction sous forme d’image de taille $T \times T \times (5 + C)$.
- Loss: dans chaque cellule, MSE pour chaque élément (par exemple $(x - \hat{x})^2, (y - \hat{y})^2 \dots$), sommé/moyenné pour toutes les cellules et toutes les images du batch

U-Net: On cherche à faire de la segmentation, donc classifier chaque pixel dans une image. Les skip connections permettent de replacer les détails de l’image, on peut y penser comme “peindre” sur l’image de base (et à différents niveaux d’abstraction).

- Architecture: encodeur-décodeur ResNet avec connections Résiduelles sur les images de même dimension/niveau de profondeur
- Loss: Cross-Entropy pour chaque pixel $(-\sum_{i: \text{pixels}} \sum_{c: \text{classes}} y_{i,c} \log \hat{y}_{i,c})$, avec $y_{i,c}$ le label de la classe c pour le pixel i .

¹ce vecteur peut s’appeler la représentation, l’embedding, les features, ou le vecteur latent en fonction de qui parle; c’est l’idée générale d’un vecteur qui encode mon input $\mathbf{x}$ dans un espace à beaucoup de dimensions.

GANs: On cherche à générer des images similaires à un dataset. Pour cela on utilise deux réseaux: un qui génère une image à partir d’une image de bruit gaussien² (générateur), et un qui devine si l’image qu’on lui montre vient du dataset ou a été générée par le générateur (discriminateur). On entraîne le discriminateur à avoir raison, le générateur à tromper le discriminateur! C’est une forme d’auto-supervisé (pas de labels, tâche prétexte).

- Architectures:

- discriminateur D : classifieur ResNet classique: image $\mathbf{x} \rightarrow$ Blocs ResNet $\rightarrow$ Couche Linéaire $\rightarrow$ image vraie ou non, $\hat{y} \in [0, 1]$
- générateur: plutôt “décodeur”, on prend une image de bruit en entrée et on propose une image en sortie: $\epsilon \rightarrow$ Blocs ResNet $\rightarrow$ image générée $\mathbf{x}_g$

- Losses:

- discriminateur: (Binary) Cross-Entropy sur sa prédiction, $-y \log \hat{y} - (1 - y) \log (1 - \hat{y})$ (même formule que Cross Entropy pour 1 seule classe vrai ou faux)
- générateur: “vrai” pour le discriminateur: minimise $-\log \hat{y}$ (maximise $\hat{y} = D(\mathbf{x}_g)$)

BERT: On cherche à apprendre un encodeur de langage. On veut faire ça sans labels $\rightarrow$ auto-supervisé. On utilise pour ça des tâches prétextes, pour lesquelles il faut comprendre le langage pour réussir. Tâches prétexte utilisées: texte à trou (MLM), cohérence d’une suite de phrases (NSP).

- Architecture: encodeur Transformer, $\mathbf{x} \rightarrow$ Bloc Transformer $\rightarrow \mathbf{z}$. Décodeur: une couche linéaire par tâche prétexte MLM: $\mathbf{z} \rightarrow$ token manquant; tâche prétexte NSP: $\mathbf{z} \rightarrow$ cohérent ou non $[0, 1]$.
- Loss: résolution de la tâche prétexte: MLM: pour une phrase $\mathbf{x}$ dont on a masqué une partie κ , $\tilde{\mathbf{x}} = \mathbf{x} \setminus \kappa$, on cherche à prédire les tokens manquants: Cross-Entropy entre le token prédit et le token à prédire, $-\sum_{t:tokens} y_t \log \hat{y}_t$. Comme il n’y en a qu’un à prédire, c’est $-\log \hat{y}_t$ à prédire, sur tout le batch.

GPT: Même combat que pour BERT, on veut un encodeur de langage, sans labels, donc auto-supervisé. Mais la tâche prétexte change: on prédit le token suivant dans la phrase/prompt.

- Architecture: pareil que BERT
- Loss: Cross Entropy entre le token prédit et le token à prédire $-\sum_{t:tokens} y_t \log \hat{y}_t$. Comme il n’y en a qu’un à prédire, c’est $-\log \hat{y}_t$ à prédire sur tout le batch.

CLIP: On veut obtenir un encodage croisé de langage ET images: pour une image et un texte qui décrit l’image, on veut que leurs vecteurs latents soient alignés. Idée critique: on veut qu’ils soient **plus alignés**³ **que les autres éléments du mini-batch!** (*contrastive*, on s’intéresse au contraste au sein du batch) C’est un problème de classification caché: laquelle de ces paires devrait être la *plus proche*?

- Architecture: encodeur de texte, encodeur d’image (Transfo, ResNet). A la fin de chaque: Couche Linéaire + Normalisation⁴ vers l’espace latent, $\mathbf{z}_t, \mathbf{z}_i$
- Loss: on veut maximiser le produit vectoriel $\mathbf{z}_t^j \cdot \mathbf{z}_i^j$ pour l’élément j du batch PAR RAPPORT au produit vectoriel de tous les indices $k \neq j$. Problème de classification caché: c’est l’élément j qui devrait être le plus proche $\rightarrow$ Cross Entropy où le “bon” indice est le j .

Diffusion et InstructGPT: Plus compliqué et **pas au programme de l’examen**.

²il faut passer un input différent à chaque fois, car bien sûr pour un input donné le générateur ne sort qu’un seul output! d’où le bruit.

³“alignés” car on parle de distance cosinus (angle) plutôt que distance L2.

⁴c’est la normalisation des $\mathbf{z}$ qui fait qu’on parle d’angle plutôt que de distance